

Temporal Cycle Enumeration for Detecting Financial Fraud

Lixiao Zheng¹, Jipeng Xiao¹, Shuai Ma^{2†}, Zuxi Chen¹, Xiangyu Luo¹

¹College of Computer Science and Technology, Huaqiao University, Xiamen 361021, China

²State Key Laboratory of Software Development Environment, Beihang University, Beijing 100191, China

Keywords: Financial fraud detection; Transaction graph; Temporal cycle; Cycle enumeration; Concurrency

Citation: Zheng L.X., Xiao J.P., Ma S., et al.: Temporal Cycle Enumeration for Detecting Financial Fraud. Data Intelligence, Vol. 7, Art. No.: 2025r19, pp. 567–590, 2025. DOI: <https://doi.org/10.3724/2096-7004.di.2024.0058>

ABSTRACT

Fraud detection is an important task of financial supervision and financial risk management. Several fraud behaviors can be connected with cycle transactions in which the money initially sent from one bank account eventually returns back to the same account. In this paper, we propose an efficient method for detecting such kind of frauds from large-scale financial transaction data. The method first constructs a transaction graph after pre-processing the original data, then divides the graph into its strongly connected components, and finally uses multiple threads to enumerate temporal cycles on different components in a parallel manner. Existing temporal cycle enumeration algorithms usually constraint the length of the cycle or the size of the time-window, which are not suitable for the specific application of financial fraud detection. In light of this, we extend the classical Johnson algorithm to enumerate temporal cycles without length and time-window constraints. To further improve the efficiency of enumeration, we introduce a block-time mechanism that avoids unnecessary multiple explorations of the same parts of the graph components. Experiments show that our method, with multithreading, is on average 15–20 times, and even 100 times faster than the existing competitor. Additionally, we adopt strategies such as amount constraint during cycle enumeration, which assist in reducing the false-positive rate of detected frauds.

1. INTRODUCTION

Digital economy has experienced a great growth in the past decade. However, with the emergence of various new digital financial payment forms, the financial criminal schemes have also evolved rapidly. For example, the Federal Trade Commission in the United States received 3.2 million fraud reports in 2019

[†] Corresponding author: Shuai Ma (E-mail: mashuai@buaa.edu.cn; ORCID: 0000-0002-4050-0443).

alone, indicating a 53% increase from 2018 [1]. Financial frauds not only bring significant economic losses to individuals and financial institutions but can also severely impact the overall development of a country's economy. Therefore, developing efficient and effective methods for detecting financial frauds is of great importance in practice.

Traditionally, financial frauds are detected manually by domain experts which are quite inefficient. In recent years, researchers have applied artificial intelligence and machine learning techniques to financial crime detection, achieving good results. For example, Teng et al. [2] proposed a framework based on Generative Adversarial Networks (GANs) for detecting online fraudsters and fraudulent transactions. The method first employs a GAN-based approach to generate training data and constructs a base pre-training fraud detection model using the generated training data. It then applies transfer learning to fine-tune the base fraud detection model using real datasets. This approach achieves a 10% improvement in precision and recall compared with the conventional methods.

Despite this, the challenge remains significant as the volume of financial data continues to grow rapidly and fraudulent activities become increasingly sophisticated. Researches show that the cost to fight and recover from financial fraud has increased by over 30% since 2016, and a large percentage is still not detected [3-4].

Several fraud behaviors can be connected with cycle transactions in which the money initially sent from one bank account eventually returns back to the same account. Existence of such cycles is a strong indicator of financial fraud such as money laundering, tax avoidance, loan fraud [5], and credit-card fraud [6]. For instance, if a group of companies makes financial transactions along a cycle through false orders and for each transaction they book false expenses, this group may try to avoid the taxation or launder illegal money. If a person uses a credit-card to fake a purchase with the help of a merchant and then the merchant sends the money back to the person after receiving payment from the bank, this person may be a credit-card "criminal".

Financial transaction data can be typically modeled as a graph structure: bank accounts are denoted as nodes, transactions between accounts are denoted as directed edges and transaction details such as time and amount serve as attributes of the edges. Consequently, the problem of detecting cycle transaction frauds reduces to detection of cycles in the transaction graph. Although cycle detection in graphs has been well studied [7-11], there are still many challenges when applied to the detection of financial frauds.

Reduction of false-positive and false-negative reports. In financial frauds, the transaction cycle usually has its own characteristics. For example, the transactions are executed in a "continuous manner" with respect to time dimension. Traditional cycle enumeration algorithms such as Johnson algorithm [7] only find simple cycles of a graph, without considering constraints such as "time increasing". This may result in a large number of cycles enumerated and thus a high false-positive rate. False-positive alarms may not only incur unnecessary investigation and audit expenses but also lead to the accusation of innocent individuals. Kumar et al [9] and Pan et al [11] study algorithms that can enumerate temporal cycles, that is, the cycle exhibits a continuous increase in time dimension. However, these algorithms are designed to

search for temporal cycles under cycle length or time-window constraints. This means that the length of the enumerated cycles—i.e., the maximum number of edges in the cycle, should not exceed a predefined value, or the duration between the start time and end time should be within a sliding time-window. However, in real-world, the entire fraudulent process can be very hidden, involving a complex chain of transactions. It is hard to predict in advance the transaction chain length or the transaction duration of a fraud. Thus applying such algorithms to identifying financial frauds may lead to false-negatives reports, that is, some fraudulent behaviors may not be successfully detected.

Dealing with large-scale transaction data. Hajdu et al [5] propose to detect suspicious fraud activity by enumerating temporal cycles without imposing restrictions on time-window and cycle length. They employ a relatively simple Depth First Search (DFS) strategy, starting from a transaction edge and performing DFS traversal to check whether a cycle returning to that transaction exists. However, this method has the following limitations. Firstly, searching from each transaction edge results in a large number of redundant checks. This may cause inefficient performance, especially when handling large financial transaction graphs. Secondly, the method enumerates non-simple cycles—i.e., the nodes in the cycle are allowed to repeat. As a result, the final enumerated cycles may contain multiple simple cycles which is not beneficial to analysis for financial experts. Some researchers consider parallel cycle enumeration [10]. However, these algorithms either do not consider temporal cycles or require length or time-window constraints, not suitable for the application of financial fraud detection scenario.

In this paper, we propose an efficient method for detecting suspicious cycle transactions from large-scale financial data. We first filter out transactions with small amounts (the threshold can be set by financial experts) considering that such transactions are unlikely to constitute frauds. We then construct the transaction graph. Next, we compute the strongly connected components (SCCs) of the transaction graph, removing transaction edges that cannot form cycles. Finally, we employ a multi-threaded concurrent approach to perform cycle enumeration on multiple SCCs simultaneously.

To reduce false-positive and false-negative reports, we propose to enumerate simple temporal cycles without length and time-window constraints. To achieve this, we extend the classical Johnson algorithm for enumerating simple directed cycles to enumerating simple temporal cycles. Similar to Hajdu [5], we also take the transaction amounts into account during cycle enumeration process. For example, the ratio between the amounts of adjacent transaction edges should not exceed 10%. This strategy further excludes cycles in which the money of each transfer differs too much, unlikely to be a suspicious activity. In such a way, the number of false-positive cycles is further reduced. In addition, we incorporate a block-time mechanism, as inspired by Kumar et al [9], into our cycle enumeration algorithm. During the DFS searching for temporal cycles, we maintain a block-time for each node already visited by now and utilize this information to prune previously unsuccessful traversal paths. This helps avoid unnecessary multiple explorations of the same parts of the SCC and thus improve algorithm efficiency.

We conduct experiments on 6 open-source datasets and compare with the method of Hajdu. The experimental results show that, when using our extension of Johnson algorithm to enumerate temporal

cycles, our method is on average 5–8 times faster than Hajdu’s with 5 threads. When using our block-time based algorithm to enumerate temporal cycles, our method is on average 15–20 times, and even 100 times for some datasets, faster than Hajdu’s. This verifies that the concurrent multi-threaded enumeration and the block-time mechanism reduces the running time greatly and make our method more practical when detecting frauds from large-scale transaction data. Experiments also demonstrate that the amount constraint imposed during cycle enumeration can effectively reduce the false-positive rate, enabling financial experts to quickly identify potential fraudulent activities.

The main contributions of this paper are summarized as follows:

1) We propose a financial fraud detection method based on concurrent temporal cycle enumeration. The method first filters out transactions with small amounts and construct the transaction graph. It then divides the graph into its SCCs and each thread performs a depth-first search on a different SCC to find cycles satisfying both time and amount constraints.

2) We extend the classical Johnson algorithm to enumerate temporal cycles without length and time-window constraints. Based on this, we introduce a block-time mechanism and propose a more efficient enumeration algorithm. This algorithm fully utilizes the timestamp attributes on transaction edges to reduce unnecessary searches during the enumeration process.

3) We implement the fraud detection method and evaluate it through experiments on open-source datasets. Experimental results demonstrate that the concurrent mechanism in our method and the block-time based temporal cycle enumeration algorithm can greatly improve fraud detection speed. Additionally, our strategies such as adopting amount constraint help reduce the number of cycles and the false-positive rate.

The structure of this paper is as follows: Section 2 introduces the relevant concepts and notions. Section 3 describes the overall framework of our method. Section 4 presents our temporal cycle enumeration algorithms. Section 5 provides experimental results. Section 6 discusses related work and finally Section 7 summarizes the paper.

2. PRELIMINARIES

2.1 Graph

A directed graph (graph for short) $G = (V, E)$ consists of a finite non-empty set of nodes V and a set of edges E representing relationships between nodes in graph G . Each edge e is an ordered pair $\langle v, w \rangle$, where $v, w \in V$, v and w are called neighbors, and e is called out-coming from v and incoming to w . The out-degree and in-degree of node v is defined to be the number of out-coming and in-coming edges of v respectively. A path p of a graph $G(V, E)$ is a sequence of nodes $v_1, v_2, \dots, v_n$, such that $\langle v_i, v_{i+1} \rangle \in E$ for $1 \leq i < n$, and v_1 is called the starting node of path p and v_n the ending node of path p . A cycle is a path where the starting and ending nodes are the same. If all nodes on a cycle, except for the starting and ending nodes, are distinct, then this cycle is called a simple cycle. For graph $G = (V, E)$ and graph $G' = (V', E')$, if $V \subseteq V'$ and $E \subseteq E'$, we say that G is a subgraph of G' .

In graph $G=(V, E)$, if there exists a path from node v_i to node v_j ($i \neq j$), and also exists a path from v_j to v_i , then v_i and v_j are said to be strongly connected. If every pair of nodes in a directed graph G is strongly connected, G is called a strongly connected graph. A maximal strongly connected subgraph of G is called a strongly connected component of G .

2.2 Problem Formulation

A financial transaction usually contains the following information: the “from” account and the “to” account, the transfer amount and the time when the transfer happens. It is natural to model the transactions as a directed graph where bank accounts are represented as nodes, transfers as directed edges, and transfer amount and timestamp serve as attributes of the edges. A suspicious transaction cycle is a temporal cycle in which the timestamp increases from the first transfer to the last transfer. Besides, the amounts on the edges in a suspicious transaction cycle are usually differ not too much from each other [5].

Formally, a financial transaction graph is a directed and temporal graph such that the edges are attributed with timestamps and amounts. In this paper, we investigate the problem of detecting cycles in a transaction graph satisfying the following rules:

- 1) The cycle is simple, that is, nodes are not allowed to repeat except for the starting one and the ending one.
- 2) Suppose that $t_0, t_1, \dots, t_n$ is the timestamp sequence of a simple cycle. Then it must satisfy that $t_i < t_{i+1}$ ($0 \leq i < n$), that is, the timestamps are in ascending order.
- 3) Suppose that a_0 is the amount of the first transaction edge in the cycle. The subsequent transaction amount a_i ($0 < i < n$) must satisfy $a_0 * (1-\theta) \leq a_i \leq a_0 * (1 + \theta)$, where $0 \leq \theta \leq 1$. That is, the difference between the amounts of transactions in the cycle is bounded by a parameter θ which can be set by the specific user requirements.

3. THE PROPOSED METHOD

The overall framework of the method proposed in this paper is shown in Figure 1. In the first stage, the original financial data is pre-processed to filter out small amounts transactions and then the corresponding transaction graph is constructed. In the second stage, all strongly connected components of the transaction graph are computed to avoid searching subgraphs with no cycles contained. In the third stage, multi-threads are employed on different strongly connected components to enumerate cycles simultaneously.

3.1 Pre-processing and Construction of Transaction Graph

The primary objective of the initial filtering is to reduce the number of transaction edges that are unlikely to be involved in fraudulent activities. Since the transaction data is massive and fraud transactions only constitute a small portion of it, removing regular legitimate transactions can significantly reduce the

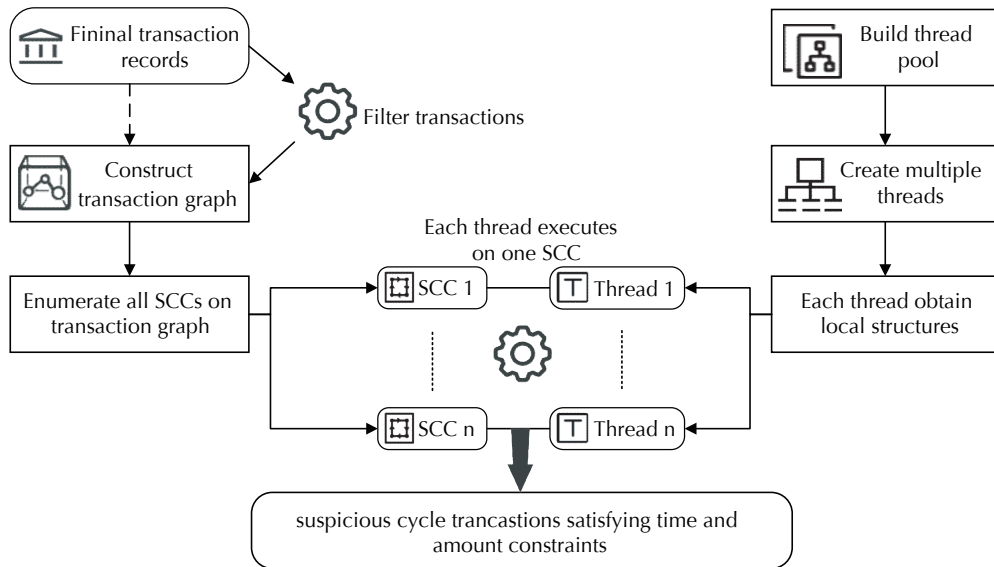


Figure 1. Overall structure of the method.

size of the transaction graph. As fraudulent activities such as money laundering often involve large-amount transfers, transaction data can be filtered according to the transfer amounts. The filtered amount can be flexibly determined by experts of the financial institutions.

After pro-processing, the remaining transaction records are used to construct the transaction graph. Considering that a client can have multiple accounts in the bank, and in the fraud detection process, the suspicious cycles should be identified between the clients and not just between the bank accounts, we propose to merge the accounts belonging to the same client into one node when constructing the transaction graph.

3.2 Computation of SCCs

The cycles of a graph are always contained within strongly connected components. Therefore, by computing all the SCCs in the transaction graph, we can identify the regions where all the simple cycles are located. This allows us to restrict the search area within each SCC. Tarjan's algorithm [12] is a seminal algorithm for computing SCCs in a directed graph. This algorithm is based on the DFS strategy, where each SCC is treated as a subtree in the searching traversal. During the depth-first search, the unvisited nodes in the current subtree are added to a stack, and during backtracking, it is checked whether the top of the stack forms a SCC with the nodes in the subtree. By performing a single traversal of the graph, Tarjan's algorithm can find all the SCCs with a time complexity of $O(n + e)$, where n and e represents the number of nodes and edges of the graph, respectively.

3.3 Concurrent Enumeration of Cycles

The traditional approach of using a single-thread serial method for cycle enumeration has many limitations. Firstly, the serial method cannot fully leverage the high concurrency of the multi-core processors, resulting in significant waste of hardware resources. Secondly, the efficiency of serial cycle detection is low, causing delays in feedback and failing to meet the real-time requirements of financial risk control. Since there is no node or edge intersections between different SCCs, using multiple threads to concurrently search cycles on multiple SCCs can not only avoids conflicts between threads but also fully utilize the concurrency of multi-core processors.

In this paper, we use thread pools to achieve multi-threaded concurrency, as shown in Algorithm 1. The input parameters are the number of threads to be created and the task set to be processed. Lines 2–4 allocate and create the specified number of threads. The task queue “*deqTask*” is used to store the set of tasks, and lines 5–7 push tasks into the queue. The lock structure in Line 8 ensures that only one thread can read tasks from the queue at any given time. Each thread takes turn to pick up a SCC and execute cycle enumeration algorithm on it.

Algorithm 1. ThreadPool (*num*, *tasks*).

Input: threads number *num*, task set *tasks*

Output: created threads and task queues *deqTask*

```
1: task ← ∅
2: FOR i = 1 TO num
3:   Apply for space and create thread;
4: END FOR
5: FOR i = 1 TO tasks.size
6:   deqTask.push(tasks);
7: END FOR
8: FOR i = 1 TO tasks.size
9:   unique_lock<mutex> lock(mutex); // Locked
10:  IF (deqTask != NULL)
11:    task = move(deqTask.front()); // Get task
12:    deqTask.pop();
13:  END IF
14: END FOR
```

4. TEMPORAL CYCLE ENUMERATION

4.1 Extension of Johnson algorithm

4.1.1 Johnson cycle enumeration algorithm

Johnson cycle enumeration algorithm [7] can enumerate all simple cycles in a directed graph, with a time complexity of $O((n + e)(c + 1))$ and a space complexity of $O(n + e)$, where n is the number of

nodes, e is the number of edges, and c is the number of simple cycles in the graph, respectively. The basic idea of Johnson algorithm is to traverse each node in the graph and attempt to enumerate all simple cycles with that node as both the start and the end nodes. When extending a path from the starting node s and adding a node v to the current path, the state of node v is set to “blocked” until all paths reachable from node v have been searched, ensuring that node v is not revisited until then. Once a simple path from s to v and back to s is found, a cascading unblock process is called. This process involves unblocking node v and all nodes blocked due to the blocking of v are unblocked. Meanwhile, the information about successfully finding a cycle is propagated back to node s during backtracking. Conversely, if no cycle is found, node v remains blocked, and all its adjacent nodes are also blocked, indicating that paths passing through v cannot form cycles. Only after node v is unblocked can its adjacent nodes be unblocked.

Next, we use an example to illustrate blocking and unblocking operations in Johnson algorithm. As shown in Figure 2(1), during the process of enumerating simple cycles starting from node a , if the search path is $a \rightarrow b \rightarrow c \rightarrow d \rightarrow e \rightarrow b$ and there is no cycle formed at node a in the end, then node b is “blocked” when accessing edge $a \rightarrow b$. Therefore, it is needed to add node e to the blocking list of node b , indicating that if node b is unblocked, node e will also be unblocked accordingly. Similarly, node d will be added to the blocking list of node e . Then, the algorithm backtracks to node d and continues new traversal from node d , resulting in the path $a \rightarrow b \rightarrow c \rightarrow d \rightarrow f \rightarrow a$. At this point, a simple cycle is detected at node a , and the algorithm will unblock node f . During the backtracking process, the information (*True*) of successfully finding a cycle is passed back to node a as a return value, and *Unblock()* is executed during the backtracking. When backtracking to node b , node b is unblocked, and all nodes blocked due to the blocking of node b are unblocked, ensuring that cycles starting from node b can be explored when the search starts from node b . Suppose that here is an edge $f \rightarrow e$ in the graph. Since after the search path $a \rightarrow b \rightarrow c \rightarrow d \rightarrow e$, node e is blocked, when a new path $a \rightarrow b \rightarrow c \rightarrow d \rightarrow f \rightarrow e$ arriving at node e , node e and all edges associated with it will not be explored again, avoiding unnecessary duplicate traversal.

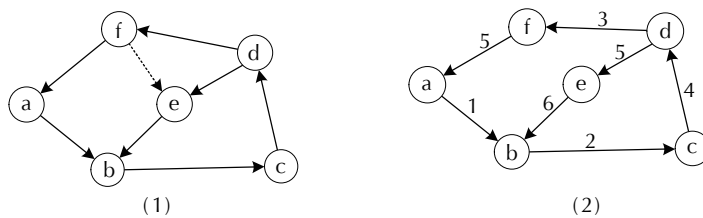


Figure 2. Example graphs for Johnson algorithm.

4.1.2 Temporal Extension of Johnson Algorithm

Johnson algorithm has high enumeration efficiency, but when extended to handle transaction graphs in which time and amount constraints need to be taken into account, it faces the following problem. The pruning method of Johnson algorithm requires traversing the entire SCC structure to accomplish “unblock” actions. However, during traversal in the transaction graph, some paths may not satisfy the time constraint or

the amount constraint. As a result, certain graph structures of the SCC can not be fully traversed. This leads to some nodes being unable to be unblocked and may cause some temporal cycles not being detected.

For example, consider Figure 2(2) where timestamps are added to the example graph in Figure 2. When search path $a \xrightarrow{1} b \xrightarrow{2} c \xrightarrow{4} d \xrightarrow{5} e \xrightarrow{6} b$, it is necessary to “block” the node e and add nodes e and d to the blocking list of node b and e respectively. However, after backtracking, when visiting $d \xrightarrow{3} f$, a problem arises because the timestamp of edge $c \xrightarrow{4} d$ is 4, which is later than the timestamp of edge $d \xrightarrow{3} f$, and clearly this path does not satisfy the strict increasing constraint on timestamps, so the traversal does not continue. At this point, since there is no cycle detected at node a , the “unblocking” operation is not executed during backtracking and thus node e remains in a “blocked” state. When searching with node b as the starting node, the temporal cycle $b \xrightarrow{2} c \xrightarrow{4} d \xrightarrow{5} e \xrightarrow{6} b$ cannot be enumerated due to the “blocking” of node e .

To prevent the above-mentioned problem and to make the most of the pruning idea of Johnson algorithm, we propose the following solution: when it encounters an edge of a node where the time constraint or the amount constraint cannot be satisfied, it traverses to the next edge of the node. If none of the edges satisfy the constraints, all “blocked” nodes on the path are forced to be unblocked.

The pseudo-code for the temporal extension of Johnson algorithm is shown in Algorithm 2. The input parameters are: the starting node s , the current node cur , the timestamp $pretime$ on the previous edge, and the amount floating range $intermoney$. Lines 1–3 initialize relevant variables, including: the cycle detection $cycle_flag$, the $constraint_mark$ variable indicating whether there are adjacent nodes satisfying the constraints, and blocking the starting node s in the $block$. Lines 4–16 conducts the depth-first search from current node cur in a recursive way. Line 6 checks if the current transaction edge satisfies both the timestamp ascending order constraint and the amount floating constraint. Lines 7–9 output the cycle and meanwhile modify the $cycle_flag$ when a cycle is found. Lines 10–13 handle the case where a cycle has not been found yet. In this case, it first set the $constraint_mark$ variable as True, marking that there exists adjacent nodes satisfying the temporal and amount floating constraints, then continues the depth-first search from node v . If none of cur ’s neighbors satisfies constraints, then $constraint_mark$ is set as True on Line 17. Lines 18–24 handle the scenario where a cycle is found. The algorithm executes *Unblock()* to unblock the current node if a cycle is formed, and if not, it blocks all adjacent nodes of cur into the block list $Blist$ of node cur . Finally, line 25 returns the boolean variable $cycle_flag$, indicating whether a cycle back to s is detected during the depth-first search from node cur .

Algorithm 2. JMTTcyc ($s, cur, pretime, intermoney$).

Input: starting node s , current node cur , timestamp $pretime$ of the previous edge, amount constraint range $intermoney$;

Output: cycle-forming information $cycle_flag$ and cycles that satisfy time and amount constraints;

```

1: cycle_flag ← false; // whether or not the markers are cycled
2: block[cur] = true;
3: constraint_mark ← false; // mark if any neighboring node satisfies the constraint
4: FOR each edge  $\langle cur, v \rangle$  of node  $cur$ 
5:   Let  $time$  and  $money$  be the timestamp and amount of edge  $\langle cur, v \rangle$ 
6:   IF ( $time > pretime \&\& money \in intermoney$ )
7:     IF ( $v == s$ )
8:       output the cycle;
9:       cycle_flag = true;
10:    ELSE IF ( $v$  is not blocked)
11:      constraint_mark = true; // marking neighbor nodes satisfy the constraint
12:      cycle_flag = JMTTcyc( $s, v, time, money$ ) || cycle_flag;
13:    END IF
14:  END IF
15: END IF
16: END FOR
17: IF ( $!constraint\_mark$ ) cycle_flag = true; END IF
18: IF ( $cycle\_flag$ ) Unblock( $cur$ );
19: ELSE FOR each neighbor node  $v$  of  $cur$ 
20:   IF ( $v$  is not visited)
21:     Blist[v].push_back( $cur$ );
22:   END IF
23: END FOR
24: END IF
25: RETURN cycle_flag;

```

Consider the example in Figure 2(2), When accessing the path $a \xrightarrow{1} b \xrightarrow{2} c \xrightarrow{4} d \xrightarrow{3} f$, the timestamps of the edge $d \xrightarrow{3} f$ is smaller than the timestamps of $c \xrightarrow{4} d$, the failure to satisfy the timestamp ascending order constraint and there are no other edges from d to f . At this point, it is assumed that a cycle has already formed at node d , corresponding to line 16 in Algorithm 3. The “True” information is then passed back to other nodes on the path through backtracking. When backtracking to node b , node e will be unblocked. Then, when starting the search from node b , the path $b \xrightarrow{2} c \xrightarrow{4} d \xrightarrow{5} e \xrightarrow{6} b$ can be enumerated. When the amount cannot satisfy the constraint, algorithm uses the similar idea to unblock nodes. However, this unblocking method is relatively simple and cannot fully utilize the timestamp information of the transaction edges, leading to efficiency issues. In view of this, we propose an improved temporal cycle enumeration algorithm, as explained in detail in the subsequent subsection.

4.2 Block-time Based Temporal Cycle Enumeration

In the temporal extension of Johnson algorithm, the timestamp attribute is only used to determine the temporal increasing constraint. Meanwhile, the blocking of nodes has only two states, either blocked or unblocked, which is too rough. To address this, we introduce the concept of “block-time” on blocked nodes. This concept is inspired by the “closing-time” mechanism in 2SCENT algorithm [9]. It utilizes the collected time information during DFS traversal to avoid unnecessary multiple explorations of the same parts of the SCCs.

Intuitively, on the path searching for cycles from the starting node s , the block-time of a node v records the maximum timestamp currently found among all the sub-paths from v which successfully form cycles back to the starting node s . In the next depth-first search arriving at node v , if the timestamp on the transaction edge is greater than or equal to the block-time of v , then it can be inferred that there does not exist temporal cycles back to s from node v . In such case, we can abort our search.

Based on the above analysis, we replace the block state used in Johnson algorithm with block-time. The basic idea of the algorithm using block-time is as follows. Starting from node s (the starting node), the algorithm begins the search in a depth-first manner. When trying to expand the depth-first path with an edge e , it firstly checks whether the timestamp Te on edge e is greater than the timestamp of the previous edge (the timestamp of the previous edge for the starting node is defaulted to -1), then determines whether Te is less than the block-time of v where v is the arriving node of edge e . If Te is greater than or equal to the block-time, the subsequent search can be stopped, and meanwhile the outcoming node w of edge e is blocked in v 's block list with time Te ; otherwise, the algorithm continues traversal from node v . If a cycle is finally found, the maximum timestamp of the cycle is used to unblock all the nodes on the path and meanwhile update the block-time of those nodes.

Different from Johnson algorithm, the block list in our algorithm contains a set of pairs which records not only the node to be blocked but also the time associated with this node when blocked. Algorithm 3 shows the details of blocking node w into the block list of node v with time Te . If w has not been yet blocked before, then adds the pair (w, Te) into the $Blist$ of v , otherwise, updates the time of w if Te is larger than the time previously added.

Algorithm 3. MakeBlist(v, w, Te).

Input: node v whose block list is to be added, node w to be blocked, time Te ;

Output: $Blist$ array after blocking;

```
1: IF ( there exists a pair  $(w, t')$  in  $Blist[v]$ 
2:   IF  $(t' < Te)$ 
3:     Replace  $(w, t')$  with  $(w, Te)$ ;
4:   END IF
5: ELSE Add  $(w, Te)$  to  $Blist[v]$ ;
6: END IF
```

Algorithm 4 shows the process of unblocking nodes on the path when a cycle is found starting from node s . T is the the maximum timestamp of the cycle, which is used to update the block-time of those nodes. Lines 1–3 first check whether the block-time of s is less than T . If yes, update it accordingly. Lines 4–11 recursively unblock each node v in the block list of s if the block-time of v is less than T . Furthermore, if the time t when node v is blocked is less than T , then delete the pair (v, t) from the block list (Lines 8–10).

Algorithm 4. Unblock (s, T) .

Input: unblock node s , unblock time T ;

Output: unblocked blist chain table;

```

1: IF (blocktime[s] < T)
2:   blocktime[s] = T;
3: END IF
4: FOR each pair  $(v, t)$  of Blist[s]
5:   IF (blocktime[v] < T)
6:     Unblock  $(v, T)$ ;
7:   END IF
8:   IF  $(t < T)$ 
9:     Delete  $(v, t)$  from Blist[s];
10:  END IF
11: END FOR

```

Considering the temporal graph shown in Figure 3(1). Starting from node a , the search proceeds by visiting edge $a \xrightarrow{1} b$. The timestamp of the previous edge from the starting node (denoted as *ptime*) is defaulted to -1 . Since the timestamp of edge from a to b is greater than *ptime*, it satisfies time constraint. The blocking time of node b is initially set to the default maximum value (MAX). Since $1 < \text{MAX}$ and b is not visited yet, the blocking time of node b is reset to 1, and the search continues from node b . Along the same lines, after traversing the path $a \xrightarrow{1} b \xrightarrow{2} c \xrightarrow{4} d \xrightarrow{5} e$, the blocking times of nodes a, b, c, d and e are MAX, 1, 2, 4 and 5, respectively. When continuing to visit $e \xrightarrow{6} b$, the blocking time of node b is 1, which is less than 6. So the extension of path stops and $(e, 6)$ needs to be added to Blist $[b]$. Then, the depth-first traversal backtracks to edge $d \xrightarrow{5} e$. Since this edge does not form a cycle, it needs to be blocked as well, So $(d, 5)$ is added to Blist $[e]$. When visiting the edge $d \xrightarrow{7} a$, the path $a \xrightarrow{1} b \xrightarrow{2} c \xrightarrow{4} d \xrightarrow{7} a$ forms a cycle, and the unblock time T is 7. Upon finding the cycle, the algorithm starts unblocking recursively

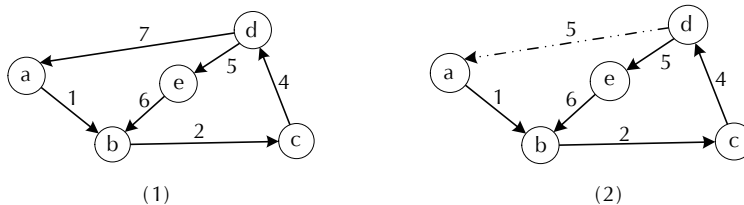


Figure 3. Examples of blocking time applications.

until reaching b . The blocking times of b , c , and d are all changed to 7. In $Blist[b]$, there is a pair $(e, 6)$. During the process of recursive unblocking, the blocking time of e is also changed to 7. When b is taken as the starting node, the cycle $b \xrightarrow{2} c \xrightarrow{4} d \xrightarrow{5} e \xrightarrow{6} b$ can be found.

The recursive depth-first search of cycles starting from node s is shown in Algorithm 5. The input parameters are as follows: the starting node s , the current node cur , the timestamp $pretime$ of the previous edge, the time T used to unblock nodes, the amount constraint range *intermoney* and the *money_mark* marking whether the finally detected cycle satisfies amount constraint. Lines 1–2 initialize the current cycle detection flag *cycle_flag* and the unblock time *unblock_time*. Lines 3–30 try to proceed the depth-first search from each adjacent edges of the current node cur . After verifying that the timestamp of the adjacent edge satisfies the time constraint in lines 6–7, two flags, *temporary_cycle_flag* and *constraint_money_mark*, need to be initialized in order to record the cycle information during backtracking and to indicate whether the amount constraint is satisfied. In lines 8–10, the algorithm checks if the amount of the current edge satisfies amount constraint. Lines 11–16 handle the case when a simple cycle is found: if the cycle satisfies the amount constraint, output it, and then the current unblock time is updated by taking the maximum value of the timestamp of the current edge and the current unblock time. In the case of a simple cycle not found (lines 17–23), if the current node is not visited yet and the timestamp *time* of the current edge is less than the blocking time *blocktime*, then the recursive search continues. Lines 24–29 check if the current edge is part of a cycle satisfying our constraints, if yes, then the cycle-forming flag and the unblock time are updated; if not, the node needs to be blocked. When backtracking, unblock function is called if a cycle is successfully found (lines 31–33).

When using block-time based algorithm to enumerate all cycles starting from a specific node, details should be paid attention to when passing parameters. As shown in Figure 3(2), if the timestamp of edge $d \xrightarrow{7} a$ is less than 6 or when it does not form a cycle, some nodes may not be successfully unblocked. This means that even if $a \xrightarrow{1} b \xrightarrow{2} c \xrightarrow{4} d \xrightarrow{5} e$ forms a cycle and the unblock time is 5, when unblocking node b , the timestamp of edge $(e, 6)$ on $Blist[b]$ list is greater than 5, preventing the block-time of e from being updated normally. When b is used as the starting node, the edge $d \xrightarrow{5} e$ cannot be accessed due to the block-time of e being 5, which causes the cycle $b \xrightarrow{2} c \xrightarrow{4} d \xrightarrow{5} e \xrightarrow{6} b$ not being enumerated. Therefore, before searching with each node as the starting node, it is necessary to unblock the nodes with the maximum value as the unblock time (as shown in Algorithm 6 line 1), to ensure that the cycles starting from this node can all be searched. For example, after searching from node a , when node b becomes the starting node, the unblocking of b will update the block-time of node e to MAX, allowing $d \xrightarrow{5} e$ to be accessed, and the cycle $b \xrightarrow{2} c \xrightarrow{4} d \xrightarrow{5} e \xrightarrow{6} b$ can be enumerated.

Algorithm 5. BMTTcyc ($s, cur, pretime, T, intermoney, money_mark$).

Input: start node s , current node cur , timestamp $pretime$ of the previous edge, unblock time T , amount constraint range $intermoney$, and $money_mark$ marking whether cycle satisfies amount constraint;

Output: cycle-forming information $cycle_flag$ and cycles that satisfy time and amount constraints;

```
1: cycle_flag ← false; // whether or not the markers are cycled
2: unblock_time ← 0; // current unblock time
3: For each edge  $\langle cur, v \rangle$  of node  $cur$ 
4:   Let  $time$  and  $money$  be the timestamp and amount of edge  $\langle cur, v \rangle$ 
5:   IF ( $time > pretime$ )
6:     temporary_cycle_flag ← false;
7:     constraint_money_mark ← false; // whether the path satisfies the amount constraint
8:     IF ( $money\_mark \parallel !(money \in intermoney)$ )
9:       constraint_money_mark = true;
10:    END IF
11:    IF ( $v == s$ )
12:      IF ( $!constraint\_money\_mark$ )
13:        Output the cycle;
14:      END IF
15:      temporary_cycle_flag = true;
16:      unblock_time = unblock_time > time? unblock_time: time;
17:    ELSE IF ( $v$  is not visited &&  $time < blocktime[v]$ )
18:      visited[v] = true;
19:      blocktime[v] = time;
20:      temporary_cycle_flag = BMTTcyc ( $s, v, time, T, intermoney, money\_mark$ )
21:        // temporary_cycle_flag;
22:      visited[v] = false;
23:    END IF
24:  END IF
25:  IF ( $temporary\_cycle\_flag$ )
26:    cycle_flag = temporary_cycle_flag
27:    T = T > unblock_time? T: unblock_time;
28:  ELSE
29:    MakeBlist( $v, cur, time$ );
30:  END IF
31: END FOR
32: IF ( $cycle\_flag$ )
33:  Unblock( $cur, T$ );
34: END IF
35: RETURN cycle_flag;
```

Algorithm 6. ALLPATH ($s, T, intermoney$).

Input: starting node s ; unblocking time T ; amount constraint range $intermoney$

Output: all temporal cycles starting from s satisfying amount constraints

```
1: Unblock ( $s, MAX$ );  
2:  $f \leftarrow false$ ; // whether the path satisfies the amount constraint  
3: BMTTcyc ( $s, -1, 0, T, intermoney, f$ )
```

5. EXPERIMENTS

5.1 Datasets

We conduct experiments on open-source financial datasets. Since there are relatively few publicly available financial dataset due to privacy issues, we add two more datasets from social networks. Social networks can be modeled as temporal graph and can be used to evaluate the temporal cycle enumeration algorithms proposed in this paper. The datasets used in our experiments are described as follows.

- 1) Codecraft1 and Codecraft2 are from the financial data used in 2020 Huawei Code Craft Competition^①. Since the original dataset is relatively small, we replicate the transactions and obtain two versions, denoted as Codecraft1 and Codecraft2.
- 2) Transactions dataset^② is from HARVARD Dataverse. This dataset contains a set of bank transaction records. The transaction amount ranges from 1 to 20000 USD.
- 3) FinalTrans dataset^③ is from Kaggle. This is a small real-world financial dataset in which the transaction amounts ranging from 1 to 13000 USD.
- 4) sx-superuser and sx-askubuntu are from Stanford large network dataset collection^④. These datasets describe interactions between users on the internet. Since most of the edges are concentrated on one or a small amount of nodes, which does not quite match the characteristics of financial data, we remove some of such edges and obtain a portion of the data for our experiments.

Table 1 illustrates the characteristics of the graph constructed from the datasets in our experiments. The datasets and the C++ implementation of our method are available online^⑤. All experiments were run on a machine with a Intel Core i5–12500 CPU @ 2.50 GHz CPU and 16 GB RAM running the Windows operating system.

^① <https://github.com/liusen1006/2020HuaweiCodecraft-TestData>.

^② <https://dataverse.harvard.edu/file.xhtml?persistentId=doi:10.7910/DVN/V6AJRV/TAD2N3>.

^③ <https://www.kaggle.com/datasets/sanskar457/fraud-transaction-detection>.

^④ <https://snap.stanford.edu/data/>.

^⑤ <https://github.com/Multithreaded-learning/A-Financial-Fraud-Detection-Method-Based-on-Temporal-Graphs>.

Table 1. Characteristics of the datasets used in experiments.

	number of nodes	number of edges	maximum out-degree	maximum in-degree
Codecraft1	40 K	1.1 M	44	36
Codecraft2	300 K	1.8 M	40	38
Transactions	83 K	530 K	1680	113
Final Trans	10 K	1.72 M	767	661
sx-askubuntu	20 K	10 K	207	105
sx-superuser	20 K	10.4 K	188	414

5.2 Performance Evaluation

We compare our method with Hajdu’s [5], the closed work to ours for detecting fraudulent cycle transactions in financial data. Since the cycles enumerated by Hajdu’s are non-simple cycles, we slightly modify it to avoid node repetition, ensuring the type of cycles same with ours for comparison. Because Hajdu’s enumeration is executed in a single thread, we use it as a baseline to compare the runtime and memory consumption of our algorithms with different thread numbers. The experimental results are shown in Figure 4–6, where Hajdu refers to the cycle enumeration algorithm of Hajdu [5], JMTTcyc refers to our extension of Johnson algorithm and BMTTcyc refers to our block-time based algorithm.

Experiments on Codecraft1 and Codecraft2 datasets. The experimental results for Codecraft1 and Codecraft2 datasets are illustrated in Figure 4. It can be observed that the single-thread version of JMTTcyc algorithm does not outperform Hajdu algorithm too much. This is probably because that the distribution of cycles in these datasets is relatively uniform. The computing of SCCs does not exclude a lot of transaction edges. Therefore, the improvement in the efficiency of a single-thread execution is not that significant. Even though, the single-thread version of BMTTcyc algorithm still has 4–5 times improvement. As the number of threads increases, the speed of enumeration is also significantly accelerated. For example, when using 15 threads, JMTTcyc algorithm is 11 and 5 times faster for Codecraft1 and Codecraft2, and BMTTcyc is even 38 and 17 times faster. In terms of memory consumption, however, our algorithms requires more space because of the auxiliary memory such as thread pools used in the multi-threaded execution. But the memory usage does not increase greatly with the addition of more threads.

Experiments on Transactions and FinalTrans datasets. The experimental results for Transactions and FinalTrans datasets are illustrated in Figure 5. At pre-processing stage, we use 900 USD and 1200 USD to filter small amounts transactions from the original two datasets respectively. As can be seen from the result, JMTTcyc does not outperform Hajdu too much in single-thread mode, but BMTTcyc is nearly 25–100 times faster than Hajdu even in single-thread version. This indicates that the pruning strategy using “block-time” in BMTTcyc is more efficient than the “either blocked or unblocked” pruning strategy. The improvement are in general increases with the number of threads. When the number of threads exceeds a

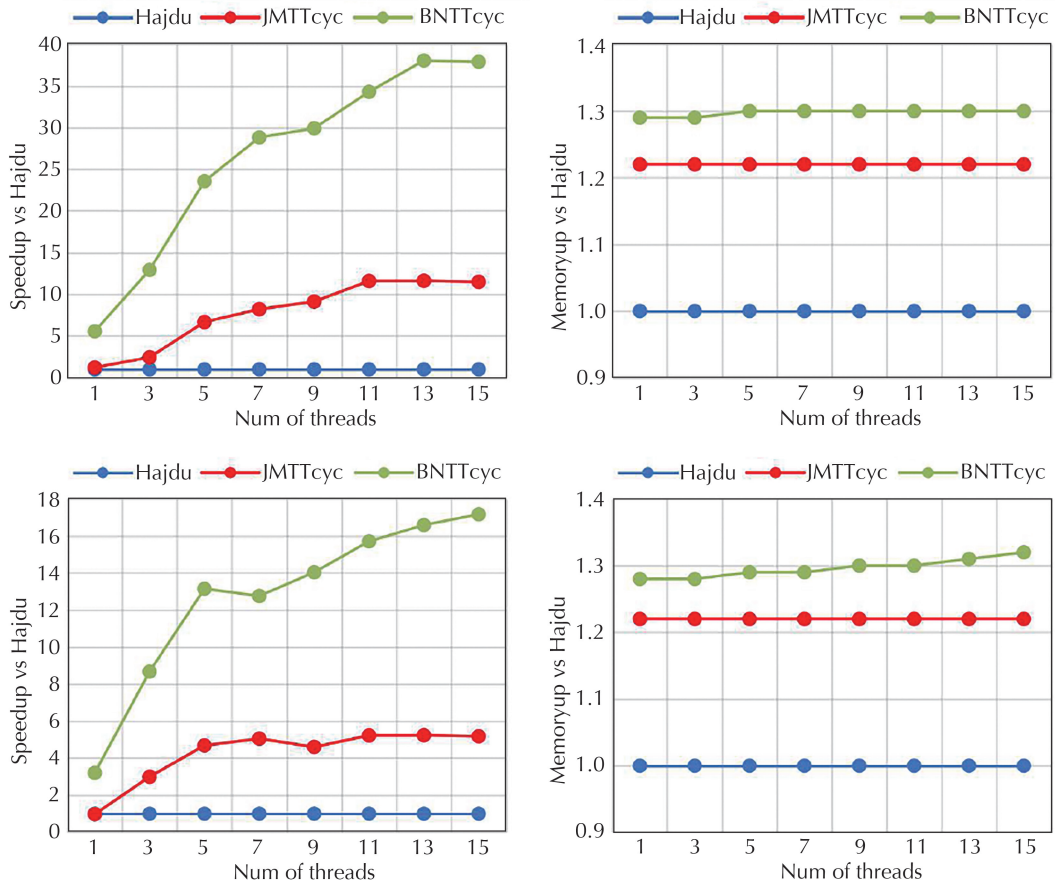


Figure 4. Execution time and memory consumption of our algorithms JMTTcyc and BMTTcyc compared with Hajdu. The above sub-figures are for Codecraft1 dataset and the below sub-figures are for Codecraft2 dataset.

certain value, however, the speedup is not that significant. This is because the concurrency of our method is limited by the number of SCCs. When the threads is more than that of SCCs, the extra threads will not help improve performance. JMTTcyc and BMTTcyc require on average 3 times more memory than Hajdu for the same reason as analyzed before.

Experiments on sx-askubuntu and sx-superuser datasets. As shown in Figures 6, our algorithms are faster than Hajdu even using only a single thread. Especially, the single-thread version of BMTTcyc is able to achieve more than 20 times speedup. This again verifies that the use of “block-time” helps improve efficiency of temporal cycle enumerating procedure. As the number of threads increases, the speedup is in general also increases. For example, when using 15 threads, JMTTcyc is 4 times faster and BMTTcyc algorithm is even 35 times faster. However, the performance increase levels out (or even drops a bit) for a certain number of threads and thereafter continues in increasing performance. We analyze that this effect likely results from the specific graph layout of sx-askubuntu and sx-superuser datasets. These two datasets

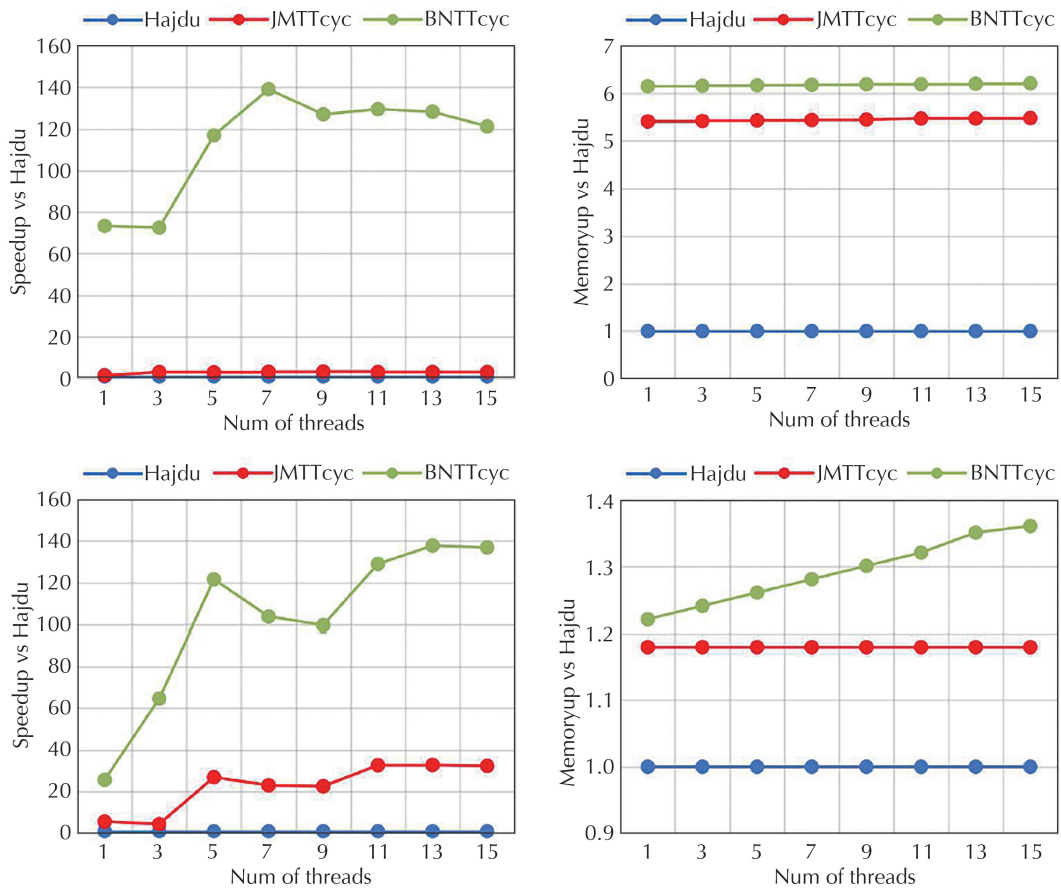


Figure 5. Execution time and memory consumption of our algorithms JMTTcyc and BMTTcyc compared with Hajdu. The above sub-figures are for Transactions dataset and the below sub-figures are for FinalTrans dataset.

come from social networks and we find that the scales of SCCs of these graphs differ too much: some SCC contain a large number of nodes and edges while some contains a relatively very small number. If a thread works on small SCC, the time consumed for communication with other threads may exceed the time saved for running the algorithm itself. So the performance increase of multi-threaded execution is not always linear. As for space consumption, JMTTcyc and BMTTcyc consumes in general more memory than Hajdu.

Table 2 shows the details of runtime when the number of threads is 7. It can be seen that both JMTTcyc and BMTTcyc have good performance on each dataset. BMTTcyc works much more better than JMTTcyc. This verifies that the concurrency enumeration strategy together with the block-time based enumeration algorithm can effectively improve fraud detection speed, making our method practical for detecting suspicious cycle transactions in large-scale financial data.

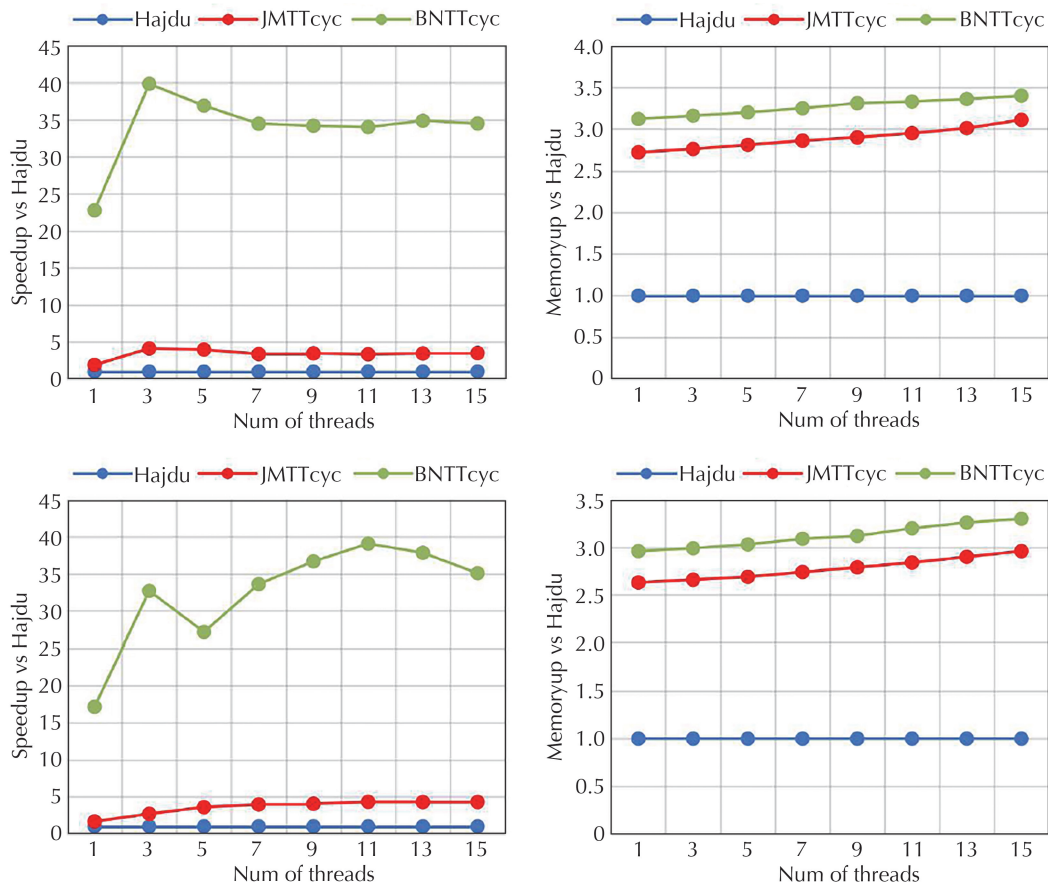


Figure 6. Execution time and memory consumption of our algorithms JMTTcyc and BMTTcyc compared with Hajdu. The above sub-figures are for sx-askubuntu dataset and the below sub-figures are for sx-superuser dataset.

Table 2. Execution time comparison of sequential Hajdu with our algorithms when using 7 threads.

Dataset	Execution time (s)			BMTTcyc Speedup vs	
	Hajdu	JMTTcyc	BMTTcyc	Hajdu	JMTTcyc
Codecraft1	18.76	2.28	0.65	28.83	3.51
Codecraft2	1308	258.5	102.5	12.76	2.52
Transactions	134.1	40.57	0.95	141.1	42.7
FinalTrans	3786	159.1	36.51	103.7	4.36
sx-askubuntu	139.6	38.65	4.05	34.47	9.54
sx-superuser	4.8	1.21	0.14	33.59	8.45

5.3 Cycle Analysis

We first investigate the effect of preliminary filtering of transactions prior to cycle enumeration. In theory, fraud transaction data occupies only a small part of the whole financial data and usually involves relatively large amount of trading money. We analyze the proportion of transactions with different amounts of the real-world financial dataset FinalTrans. As can be seen from Figure 7, the number of transactions shows a clear downward trend as the trading amount rises. So before cycle enumeration, it is necessary to exclude transactions by setting a money filtering threshold. This can reduce the search space for detecting suspicious cycle transactions.

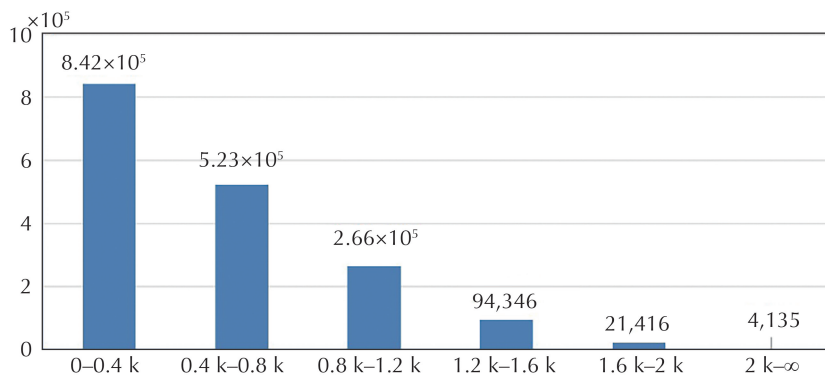


Figure 7. Number of transactions with different amounts of FinalTrans dataset.

We also investigate the effect of amount constraint on the cycles enumerated. Table 3 shows the number of suspicious cycles when using different amount constraints and different amount filtering thresholds on FinalTrans dataset. We observe that under the same filtering value, the amount constraint lead to a significant difference in the number of enumerated cycles. When the constraint is fixed, for example, the changing of the transaction amount does not exceed 5%, the number of cycles is reduced greatly. The constraint can be flexibly set by financial fraud detection experts and helps decrease the false-positive report rate, enabling financial experts to efficiently identify potential fraudulent activities.

In addition, we also observe that the number of cycles enumerated by different amount constraints differs by nearly ten times for the same filtering amount, but the number of nodes involved does not increase significantly. This suggests that real-life transactions between nodes are dense with different money amount, but the nodes (bank accounts) involved in fraudulent activities are almost stable. However, it should be noted that with a filtering amount of 600 and an amount constraint of 10%, millions of cycles are reported, which is obviously not realistic for financial experts to analyze. This suggests that the initial filtering amount and the amount constraint should to be used together to achieve better results.

Table 3. Number of nodes involved and cycles after Initial Filtering and Amount Restriction.

Filtering amount	Amount constraint	Number of cycles	Number of nodes involved
600	5%	10704	3195
	8%	531299	3538
	10%	7258459	3575
800	5%	2871	2304
	8%	69343	2952
	10%	508322	3044
1000	5%	528	995
	8%	4715	2060
	10%	18736	2343

6. RELATED WORK

Financial fraud detection. There are various types of fraudulent activities, including creditcard fraud [13], online lending fraud [14] and money laundering fraud [15] among others. In recent years, researchers have applied machine learning [16-17], graph analysis [14, 18-19] and other techniques [20-21] to financial fraud detection, achieving good results. Among them, graph-based methods model financial data using graph structures, and has become a hot research topic in the field of financial fraud detection. Wang et al. propose a semi-supervised graph attention neural model which makes full use of the relationships and attributes of labeled and unlabeled data to enhance the user information and obtain comprehensive fraud detection results [22]. Wu et al. develop a multi-task learning framework which learns from two domains, fraudulent enterprises and transactions, to increase the accuracy of fraud prediction [23]. Jamshidi et al. propose a multi-objective approach for identifying risky regions of money laundering by the inference engine of the neuro-fuzzy inference system [24]. In response to the increasing frequency and complexity of fraudulent activities, Huang et al. propose a k-means clustering method to identify abnormal patterns and behaviors to discover potential fraud [25]. Singh et al. explore the use of graph visualization techniques that help investigators “see” bank transaction data for a target entity and then use their senses to visually detect unusual patterns of suspicious money laundering activities [26]. Our work in this paper focuses mainly on detecting fraudulent cycle transactions by enumerating simple cycles satisfying both time and amount constraints.

Cycle enumeration. Cycle enumeration of graphs is a traditional problem in computer science. Most of the algorithms are based on depth-first search, which recursively searches from one node to find all cycles ending with that node. Tiernan use an exhaustive search strategy to enumerate simple cycles, but the time required for enumeration is exponential [27]. Tarjan introduce the look-ahead and labeling method to

reduce unnecessary searches based on the backtracking process of Tieran's algorithm [12]. Johnson executes the whole cycle detecting process in the strongly connected component (SCC) and performs pruning by a special block/unblock method [7]. These algorithms achieve a lower time complexity of $O((n + e)(c + 1))$ where n , e , c represents the number of nodes, edges, simple cycles of the graph respectively. The pruning method in this paper is borrowed from Johnson's algorithm, but the pruning conditions are improved accordingly. Kumar et al. introduce a naive algorithm for enumerating temporal cycles [8]. In a follow-up work [9], the same author propose 2SCENT, an improved algorithm searching for temporal cycles within a time-window. This algorithm first collects candidate root nodes for cycles and then performs a depth-first search with backtracking. They introduce a closing-time concept to prune unsuccessful depth-first traversal paths and a path-bundle concept to further reduce the number of depth-first traversal paths. Pan et al. add a length restrict on 2SCENT algorithm [11]. Our enumeration algorithm adopts the idea of closing-time from 2SCENT but does not impose time-window constraint or cycle-length constraint. Similar to our work, Hajdu et al. discuss the enumeration of temporal cycles for detecting financial fraud. But the algorithm is based on a simple depth-first search working in a serial mode which does not scales well.

Concurrent cycle enumeration. Lu et al [28] and Cui et al. [29] study multi-threaded algorithms for finding or removing simple cycles of a directed graph. The algorithms divide the graph into its SCCs and each thread performs a depth-first search on a different SCC component to find cycles. This concurrency strategy is similar to ours. Benachour et al. propose a GPU-based parallel enumeration by performing multi-threaded breadth-first search on strongly-connected components [30]. Qing et al. [31] introduce a fine-grained parallel cycle enumeration algorithm which starts searching length-constraint cycles from the same vertex in parallel. Jovan et al. observe that the efficiency of parallel searching on multiple SCCs is limited by the number of SCCs. So the authors modify classical cycle enumeration algorithms to exploit much finer-grained parallelism [10]. The detected cycles can be either simple or temporal, with time-window constraints. In this paper, we concurrently enumerate temporal cycles with no time-window or cycle-length constraints. Meanwhile, we add amount constraint on the enumerated cycles in order to reduce false-positive reports when applied to detecting financial frauds.

7. CONCLUSION

Financial frauds not only bring significant economic losses to individuals and financial institutions but can also severely impact the overall development of a country's economy. Several fraud behaviors can be connected with cycle transactions in which the money initially sent from one bank account returns back to the same account. However, traditional cycle enumeration algorithms have certain limitations in identifying such kind of fraudulent activities. In this paper, we propose a fraud detection method based on concurrent temporal cycle enumeration in a transaction graph. We adopt a multi-threaded concurrent cycle enumeration strategy to analyze large-scale transaction data and detect potential suspicious cycle transactions. We extend Johnson cycle enumeration algorithm and furthermore propose a fast block-time based algorithm to detect cycles with time and amount constraints but without length or time-window restrictions. The experimental results show that the proposed method can efficiently improve the detection

speed and reduce the false-positive rate, which helps financial experts identify potential fraudulent behaviors over large-scale financial data.

The concurrency mechanism in this paper is relatively coarse-grained and may lead to a workload imbalance. As a future work, we plan to investigate fine-grained concurrent temporal cycle enumeration which runs on one strongly connected component. In addition, the real-world fraudulent activities may be more sophisticated. For example, a large amount of money may be divided into several transaction paths and the difference of amounts between each transaction path may also be relatively large. How to detect such more complicated financial fraud also requires indepth research.

ACKNOWLEDGEMENTS

This work was supported by National Natural Science Foundation of China (61925203), Natural Science Foundation of Xiamen (3502Z20227191), Fundamental Research Funds for the Central Universities (ZQN-1010), and Natural Science Foundation of Fujian Province (2023J01137, 2021J01316).

REFERENCES

- [1] Federal Trade Commission (FTC), "Consumer Sentinel Network," Jan. 2020. [Online]. Available: <https://www.ftc.gov>.
- [2] H. Teng, C. Wang, Q. Yang, X. Chen, and R. Li, "Leveraging adversarial augmentation on imbalance data for online trading fraud detection," *IEEE Trans. Comput. Social Syst.*, vol. 11, no. 2, pp. 1602–1614, 2024.
- [3] M. Shepard, T. Adams, A. Portilla et al., *The Global Framework for Fighting Financial Crime: Enhancing Effectiveness & Improving Outcomes*. Deloitte, 2019.
- [4] LexisNexis, "2019 True Cost of Fraud Study: E-Commerce/Retail Edition". LexisNexis, 2019.
- [5] L. Hajdu and M. Kresz, "Temporal network analytics for fraud detection in the banking sector," in *Proc. Int. Conf. Theory Pract. Digit. Libr.*, 2020, pp. 145–157.
- [6] X. Qiu, W. Cen, Z. Qian et al., "Real-time constrained cycle detection in large dynamic graphs," *Proc. VLDB Endow.*, vol. 11, no. 12, pp. 1876–1888, 2018.
- [7] D. B. Johnson, "Finding all the elementary circuits of a directed graph," *SIAM J. Comput.*, vol. 4, no. 1, pp. 77–84, 1975.
- [8] R. Kumar and T. Calders, "Finding simple temporal cycles in an interaction network," in *Proc. TD-LSG@PKDD/ECML*, 2017, pp. 3–6.
- [9] R. Kumar and T. Calders, "2scent: An efficient algorithm to enumerate all simple temporal cycles," *Proc. VLDB Endow.*, vol. 11, no. 11, pp. 1441–1453, 2018.
- [10] J. Blanus, P. lenne, and K. Atasu, "Scalable fine-grained parallel cycle enumeration algorithms," in *Proc. ACM Symp. Parallelism Algorithms Archit.*, vol. 34, 2022, pp. 247–258.
- [11] M. J. Pan, R. H. Li, Y. H. Zhao, and G. R. Wang, "Fast temporal cycle enumeration algorithm on temporal graphs," *J. Softw.*, vol. 31, no. 12, pp. 3823–3835, 2020 (in Chinese).
- [12] R. Tarjan, "Enumeration of the elementary circuits of a directed graph," *SIAM J. Comput.*, vol. 2, no. 3, pp. 211–216, 1973.
- [13] S. Shirgave, C. Awati, R. More et al., "A review on credit card fraud detection using machine learning," *Int. J. Sci. Technol. Res.*, vol. 8, no. 10, pp. 1217–1220, 2019.

- [14] S. Zhao, Y. Huang, L. Chen et al., "Loan fraud users detection in online lending leveraging multiple data views," in *Proc. AAAI Conf. Artif. Intell.*, vol. 37, no. 4, 2023, pp. 5428–5436.
- [15] J. J. Xu, D. Chen, M. Chau et al., "Peer-to-peer loan fraud detection: Constructing features from transaction data," *MIS Q.*, vol. 46, no. 3, 2022.
- [16] M. Habibpour, H. Gharoun, M. Mehdipour et al., "Uncertainty-aware credit card fraud detection using deep learning," *Eng. Appl. Artif. Intell.*, vol. 123, p. 106248, 2023.
- [17] S. R. Bin, V. Schetinin, and P. Sant, "Review of machine learning approach on credit card fraud detection," *Hum.-Centric Intell. Syst.*, vol. 2, no. 1, pp. 55–68, 2022.
- [18] X. Li, S. Liu, Z. Li et al., "Flowscope: Spotting money laundering based on graphs," in *Proc. AAAI Conf. Artif. Intell.*, vol. 34, no. 4, 2020, pp. 4731–4738.
- [19] E. Kurshan, H. Shen, and H. Yu, "Financial crime & fraud detection using graph computing: Application considerations & outlook," in *Proc. IEEE Int. Conf. Transdiscipl. AI (TransAI)*, 2020, pp. 125–130.
- [20] R. J. Bolton and D. J. Hand, "Statistical fraud detection: A review," *Stat. Sci.*, vol. 17, no. 3, pp. 235–255, 2002.
- [21] J. West and M. Bhattacharya, "Intelligent financial fraud detection: A comprehensive review," *Comput. Secur.*, vol. 57, pp. 47–66, 2016.
- [22] D. Wang, Y. Qi, J. Lin et al., "A semi-supervised graph attentive network for financial fraud detection," in *Proc. IEEE Int. Conf. Data Mining (ICDM)*, 2019, pp. 598–607.
- [23] B. Wu, K. Chao, and Y. Li, "DualFraud: Dual-target fraud detection and explanation in supply chain finance across heterogeneous graphs," in *Proc. Int. Conf. Database Syst. Adv. Appl.*, 2023, pp. 370–379.
- [24] M. B. Jamshidi, M. Gorjiankhanza, A. Lalbakhsh et al., "A novel multiobjective approach for detecting money laundering with a neuro-fuzzy technique," in *Proc. IEEE Int. Conf. Netw., Sens. Control (ICNSC)*, vol. 16, 2019, pp. 454–458.
- [25] Z. Huang, H. Zheng, C. Li et al., "Application of machine learning-based k-means clustering for financial fraud detection," *Acad. J. Sci. Technol.*, vol. 10, no. 1, pp. 33–39, 2024.
- [26] K. Singh and P. Best, "Anti-money laundering: Using data visualization to identify suspicious activity," *Int. J. Account. Inf. Syst.*, vol. 34, p. 100418, 2019.
- [27] J. C. Tiernan, "An efficient search algorithm to find the elementary circuits of a graph," *Commun. ACM*, vol. 13, no. 12, pp. 722–726, 1970.
- [28] W. Lu, Q. Zhao, and C. Zhou, "A parallel algorithm for finding all elementary circuits of a directed graph," in *Proc. Chin. Control Conf. (CCC)*, vol. 37, 2018, pp. 3156–3161.
- [29] H. Cui, J. Niu, C. Zhou, and M. Shu, "A multi-threading algorithm to detect and remove cycles in vertex- and arc-weighted digraph," *Algorithms*, vol. 10, no. 4, p. 115, 2017.
- [30] A. Benachour, S. Yahiaoui et al., "Fast parallel algorithms for finding elementary circuits of a directed graph: A GPU-based approach," *J. Supercomput.*, vol. 79, no. 5, pp. 4791–4819, 2023.
- [31] Z. Qing, L. Yuan et al., "Efficient parallel cycle search in large graphs," in *Proc. Database Syst. Adv. Appl.*, vol. 25, no. 2, 2020, pp. 349–367.